CAMS-Bot: A Self-Balancing Two-Wheeled Cargo Platform for Users with Mobility Limitations

Jason Rupp
Department of Electrical and Computer Engineering
Northeastern University
Seattle, WA, USA
rupp.j@northeastern.edu

***Abstract*—Mobility-limited users in residential environments face a routine but high-leverage problem: transporting everyday items between rooms while their hands are needed for stability aids. Existing solutions either require the user to push the device or are sized and priced for industrial settings. This report describes CAMS-Bot, a residentially-scaled self-balancing two-wheeled cargo platform developed for the EECE 5552 Assistive Robotics course as a step toward closing this gap. The system is built around a Raspberry Pi 4 running ROS2 Jazzy in Docker for high-level control, a Raspberry Pi Pico running a cascaded PID balance controller in MicroPython at 100 Hz, an MPU6050 IMU, two 12 V DC gearmotors with quadrature encoders driving 8-inch polyurethane wheels through an L298N motor driver, and a hybrid chassis of laser-cut acrylic decks, 3D-printed PLA brackets, and PETG bushings at the wheel-motor coupling. Operator input is provided through an Xbox controller and a ROS2 teleoperation node communicating with the Pico over hardware UART. The delivered system balances reliably on residential carpet, accepts smooth teleoperated drive and turn commands, and holds the project's headline 5 kg payload in static balance. Dynamic loaded locomotion at 5 kg fails for analyzed geometric reasons, and the originally proposed autonomous person-following capability was descoped during the project. The contribution is a working proof-of-concept platform with a fully documented controller, identified failure modes, and a concrete architectural roadmap to a deployable follow-on build.**

I. Introduction

A. The assistive task and its users.

Many people who live with mobility limitations struggle to transport everyday items. For an impaired user, some transport tasks can be slow, painful, unsafe, or impossible without assistance. The assistive task addressed by this project was to build a small mobile robot that carries cargo on behalf of such a user and travels with them through a residential environment, freeing both of the user's hands for stability aids and reducing the load they must support themselves.

B. Why the task matters.

Cargo carrying is a routine but high-leverage problem for the target population. Commercial service robots designed for cargo transport are sized and priced for warehouses and hospitals rather than apartments. A compact, low-cost, residentially scaled cargo robot that follows its user would meaningfully extend independent living, and the absence of such a device on the consumer market reflects the difficulty of building one well rather than a lack of need.

C. What makes the task technically challenging.

The technical challenge is unusually broad for a residential robot. The platform had to be small enough to maneuver around furniture, which pushed the design toward a two-wheel self-balancing form factor rather than a stable four-wheel base, and self-balancing introduces a real-time control problem on top of every other subsystem. The robot also had to carry a meaningful payload (the project targeted 5 kg, sufficient for the majority of personal cargo tasks), which interacts with the balance problem because added mass changes the chassis's effective inertia and center of gravity. It needed to operate on common residential floor surfaces, such as carpet, hardwood, and tile, each of which presents different friction and rolling resistance that affect closed-loop stability. Finally, the long-term vision required onboard person tracking and following, which adds a perception pipeline running concurrently with a hard real-time balance loop on cost-constrained compute. Solving all of these problems simultaneously on a student budget was the core challenge of the project.

D. The system that was built.

CAMS-Bot (CArry My Stuff Bot) is a self-balancing two-wheeled cargo platform with a custom 3D-printed and laser-cut acrylic chassis, two 12 V DC gearmotors with quadrature encoders driving 8-inch polyurethane wheels through an L298N H-bridge, an MPU6050 IMU for tilt sensing, a Raspberry Pi 4 running ROS2 Jazzy for high-level control, and a Raspberry Pi Pico running a dual-loop PID balance controller. Drive commands are issued from an Xbox controller through a ROS2 teleoperation node on the Pi 4 and transmitted to the Pico over a hardware UART link. The system was originally scoped as follower bot with OAK-D Pro vision and YOLO-based person tracking. The delivered system balances reliably on carpet, accepts teleoperated drive, and successfully supports the 5 kg target payload during evaluation. The remainder of this report is organized as follows. Section 2 reviews related work. Section 3 describes the system as built. Section 4 presents the experimental evaluation, including PID tuning and load testing. Sections 5 and 6 discuss outcomes and lessons learned. Section 7 outlines limitations and future work.

II. Related Work

CAMS-Bot sits at the intersection of three research areas: assistive mobile robotics for users with disabilities and mobility limitations, two-wheeled self-balancing platforms in the controls literature, and commercial autonomous cargo and human-following robots. This section reviews each area, identifies the gap CAMS-Bot targets, and positions the delivered system relative to prior work.

A. Target User and Context

The assistive function targeted by CAMS-Bot serves a large and well-documented population. The Centers for Disease Control and Prevention reports that disability affects a substantial fraction of the U.S. adult population, and mobility limitations are among the most common categories of disability [1]. National-level statistics on the use of mobility devices show that millions of Americans rely on these devices in daily life [2]. For this population, even routine in-home tasks that involve carrying objects compound an existing physical burden, as users must already manage a stability aid, a chronic condition, or both.

Within assistive robotics, the broad goal of empowering users with disabilities through robotic assistance is established in the literature. Chen et al. articulated this vision under the banner of "robots for humanity," demonstrating that mobile manipulation systems can extend independence for users with motor impairments [3]. More targeted work in the same space includes Balatti et al., who developed a robot-assisted navigation system for visually impaired users using adaptive impedance and path planning [4], and Deb et al., who designed and controlled an autonomous robot specifically for mobility-impaired patients [5]. Recent work by Olatunji et al. on the Stretch platform examines robotic support for older adults with combined cognitive and mobility impairments [6]. Common metrics for evaluating human-robot interaction in this space, including success rates, task completion times, and user-reported workload, are reviewed by Steinfeld et al. [7] and operationalized through standard instruments such as NASA-TLX [8] and SUS variants [9].

CAMS-Bot's contribution within this body of work is narrower than mobile manipulation or visual navigation. The platform addresses one specific assistive function, transporting cargo alongside a user, on a residentially-scaled chassis, leaving the user's hands free for stability aids.

B. Existing Technical Approaches

Existing approaches relevant to CAMS-Bot fall into three categories: non-robotic mobility and carrying aids, commercial autonomous cargo and follower robots, and academic two-wheeled self-balancing platforms.

C. Non-robotic mobility aids.

Walkers, rollators, and wheeled shopping carts provide a small carrying surface for in-home and community cargo, but they require the user to push the device themselves. They provide no autonomy, no automatic following, and no mechanical assistance for the carrying task itself. For users whose strength does not support pushing a loaded device, or who need both hands free for a stability aid, these devices are inadequate.

D. Commercial autonomous cargo and follower robots.

Commercial products in the human-following cargo category exist but are designed for industrial rather than residential contexts. The Foxtech FOLO-200, for example, is a four-wheel cargo robot rated for 100 kg payload and explicitly marketed for industrial transport and outdoor logistics [10]. Its scale, four-wheel base, and outdoor sensor stack are appropriate for warehouse and material-handling environments but not for the indoor residential use case CAMS-Bot targets, where the c hassis must navigate furniture and doorways. More broadly, commercial outdoor delivery robots have a documented record of safety problems in shared pedestrian environments. News reports from 2023 onward describe a sidewalk delivery robot that lost control and damaged residential property in East Hollywood [11], a delivery robot that repeatedly struck a man with cerebral palsy using a mobility scooter in West Hollywood [12], and a delivery robot that collided with an ambulance in Hollywood [13]. These incidents underscore that even well-funded commercial cargo robots do not yet operate safely around the very population that CAMS-Bot is intended to serve, and they motivate the design choice to keep the user as the direct operator of the platform rather than ceding control to a fully autonomous outdoor stack.

E. Academic and practical two-wheeled self-balancing robots.

The two-wheeled self-balancing robot is a well-studied control problem, modeled as an inverted pendulum on a moving cart. PID, LQR, and hybrid control strategies have been evaluated extensively in the academic literature. Petcu and Andreescu compare PID and LQR controllers on a wheeled inverted pendulum and characterize the tradeoffs between the two approaches [14]. Saha et al. extend the platform with deep reinforcement learning alongside a PID baseline in their "Epersist" framework [15]. Alongside the academic literature, practical builds in the maker community have demonstrated that competent balance is achievable with low-cost microcontroller hardware, an inexpensive MEMS IMU, encoder-equipped DC gearmotors, and PID control. The CAMS-Bot balance controller is most directly informed by one such build: the widely-cited Instructables "Arduino Based Self-Balancing Bot," which provides a complete reference design including the cascaded PID structure (an inner tilt loop and an outer position or velocity loop), the MPU6050 IMU integration pattern, and a worked PID tuning sequence [16]. The CAMS-Bot implementation follows this template, with the following adaptations: the microcontroller is a Raspberry Pi Pico rather than an Arduino, the controller is written in MicroPython rather than the Arduino C dialect, the inner balance loop r uns at 100 Hz with the outer velocity loop running at a 10× lower rate via loop division, the motor driver is an L298N driven at approximately 20 kHz PWM, and the platform is

scaled up to carry a 5 kg payload on 8-inch wheels rather than the small-form-factor demonstrator of the original build. Section 3.4 describes these adaptations and the dual-loop control architecture in detail.

III. System Design

A. Design Requirements

TABLE I
Proposal Requirements and Delivery Status

Requirement	Status	Ref.
Functional		
Carry payloads up to 5 kg while balancing	Met	§4
Autonomously follow user at 0.5–1.5 m/s	Descoped	§3.4, §7
Maintain following distance of 0.5–1.0 m	Descoped	§3.4, §7
Detect and stop for obstacles in path	Descoped	§3.4, §7
No tracking device required on the user	Descoped	§3.4, §7
Usability		
Manual override via gamepad controller	Met	§3.4
Fit through standard residential doorway	Met	§3.2
Safety		
Cut motor output if tilt exceeds threshold	Partial	§3.4, §5
Stop on loss of user detection	Descoped	§3.4, §7
Cap autonomous following speed at 1.5 m/s	Descoped	§3.4, §7

The functional requirements are split between a delivered hardware-and-control layer and a descoped vision-and-tracking layer. The 5 kg payload requirement, which was the headline assistive specification of the proposal, was satisfied during evaluation; the load test is described in Section 4. The remaining four functional requirements (autonomous following, distance keeping, obstacle response, and untethered tracking) all depend on the OAK-D Pro vision and DepthAI person-detection pipeline that was the responsibility of the project partner. That subsystem was not delivered, and consequently those requirements are reported as descoped rather than partial: no portion of the autonomous-following pipeline runs on the delivered system. The hardware required to support the descoped capability (the OAK-D Pro camera, its mount, and a wired connection to the Pi 4) is in place, and the Pi 4 to Pico command interface that an autonomous-following node would use is implemented and exercised by the manual-control node. Section 3.4 describes that interface, and Section 7 describes the work required to complete the autonomous-following capability. The usability requirements are met. The manual override mode operates through an Xbox controller connected to the Raspberry Pi 4, with commands forwarded to the Pico over hardware UART (Section 3.4). The chassis dimensions fit through a standard residential doorway. The safety requirements are partially met. The proposal's first safety requirement specified an automatic motor cutoff if the tilt angle exceeds a safe threshold. As delivered, the system instead provides an operator-initiated kill switch that cuts power to the motors when pressed. This satisfies the requirement's intent for the operator-in-the-loop case (the operator can disable the motors at any time), but it does not satisfy the original automatic-cutoff specification: a fall-class tilt event completes in well under a second, faster than an operator can be expected to react. An automatic tilt-threshold cutoff in the Pico balance loop is the natural extension and is identified as future work in Section 7. The remaining two safety requirements depend on the descoped vision subsystem. A stop-on-loss-of-detection behavior cannot exist without a detection pipeline, and a cap on autonomous following speed has no application when no autonomous mode is delivered. Both are reported as descoped rather than violated, since the conditions under which they would apply are not present in the delivered system. In the manual-control configuration, speed is governed by the operator and by the balance controller's saturation behavior. A note on what is not in this table. The proposal did not separately list a requirement for stable balance across residential floor surfaces; balance was implicit in the functional requirements. As built, balance is reliable on carpet and adequate on smooth flooring such as hardwood and tile. This behavior is not a violation of any tabled requirement, but it is a real limitation of the delivered system and is discussed honestly in Section 4 (PID tuning) and Section 6 (lessons learned).

B. Robot Platform

CAMS-Bot is a self-balancing two-wheeled differential-drive cargo platform. Figure 1 shows the assembled robot, and Figure 2 shows the system block diagram. The platform is built around four design choices that follow from the requirements in Section 3.1: a two-wheel inverted-pendulum form factor for residential maneuverability, 8-inch polyurethane wheels for traction on residential flooring, a split compute architecture that isolates real-time control from high-level processing, and a 12 V power bus sized for the motor and compute loads.

Drivetrain. The drivetrain consists of two DFRobot FIT0186 12 V brushed DC gearmotors rated at 251 RPM, each with an integrated quadrature encoder. The motors are coupled to 8-inch polyurethane wheels through custom 3D-printed PETG hub adapters, described in Section 3.3. Polyurethane was selected for the wheel material because it provides higher friction on hardwood and tile than rubber-tread alternatives at the same wheel diameter, while remaining quiet on hard surfaces. The motors are driven by an L298N dual H-bridge motor driver. The L298N replaced an earlier TB6612FNG selection during integration after the TB6612FNG produced reliability problems under load; the L298N tolerated the stall and start-up currents of the 12 V gearmotors more robustly. Both motor channels are driven with PWM at approximately 20 kHz, which sits above the audible range and produces smoother current delivery to the motor inductance than lower switching frequencies.

Sensing. Tilt sensing for the balance loop is provided by an MPU6050 inertial measurement unit, which integrates a three-axis accelerometer and a three-axis gyroscope on a single I^2C device. The MPU6050 is mounted rigidly to the chassis with its sensitive axis aligned to the chassis pitch axis. Tilt angle is estimated by a complementary filter that fuses the gyroscope rate (good for short-term dynamics, prone to long-term drift) with the accelerometer-derived angle (good for long-term reference, sensitive to lateral acceleration noise). Wheel motion is sensed by the integrated quadrature encoders on each gearmotor; the Pico decodes the encoder edges in software to obtain wheel velocity for the outer velocity loop.

Compute. The compute stack is split across two boards with deliberately separated responsibilities. A Raspberry Pi Pico runs the real-time balance control loop at 100 Hz. The Pico reads the IMU, decodes the encoders, runs the cascaded PID controller, and writes PWM commands to the L298N. A Raspberry Pi 4 hosts the high-level stack: Ubuntu, ROS2 Jazzy in Docker, the Xbox-controller joystick node, the teleoperation command publisher, and the serial bridge to the Pico. The two boards communicate over a hardware UART link using a simple ASCII protocol described in Section 3.4. Splitting the stack this way isolates the deterministic balance loop from the non-deterministic timing of Linux, ROS2, and any future perception pipeline.

Vision hardware. An OAK-D Pro depth camera is mounted at the top of the chassis on a custom 3D-printed mount. The camera connects to the Raspberry Pi 4 over USB. As discussed in Section 3.1, no vision software is integrated in the delivered system; the camera and its mount are present so that the planned DepthAI and YOLO-based person-tracking pipeline could be added without further mechanical or hardware work.

User input and emergency stop. An Xbox controller is the user's primary input device, connected to the Raspberry Pi 4 over bluetooth. Drive and turn commands are mapped from the controller's analog sticks (left-stick Y for forward and back, right-stick X for turning) and forwarded over UART to the Pico. The platform also has a hardware kill switch that cuts power to the motors when pressed, providing the operator with an immediate physical override. The kill switch is the safety mechanism described in Section 3.1; an automatic tilt-threshold cutoff in the balance controller is identified as future work in Section 7.

Power. The platform uses a split power architecture. A 12 V DC battery feeds the L298N motor driver and supplies both motor channels. The Raspberry Pi 4 and the Raspberry Pi Pico are powered separately from a USB battery pack (Anker), which provides a regulated 5 V logic supply that is electrically isolated from the motor bus on the high side. The motor encoders are powered from the L298N's onboard 5 V regulator output, so encoder logic levels are referenced to the motor-side ground. Logic and motor grounds are tied together at a single point so that the Pico's encoder inputs and PWM outputs share a common reference with the motor driver. Separating the logic and motor supplies in this way prevents motor inrush and stall currents from sagging the Pi 4's rail and crashing the high-level stack, which is a known failure mode when both subsystems share a single battery.

Form factor. The chassis dimensions allow the robot to pass through a standard residential doorway. The cargo platform is integrated into the upper body of the chassis above the wheel axle and is sized to carry the 5 kg target payload.

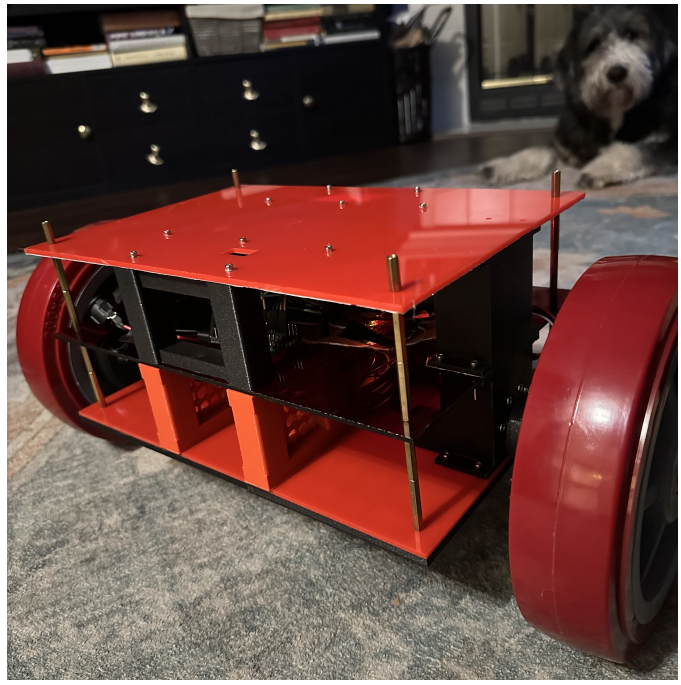

Fig. 1. CAMS-Bot, as built.

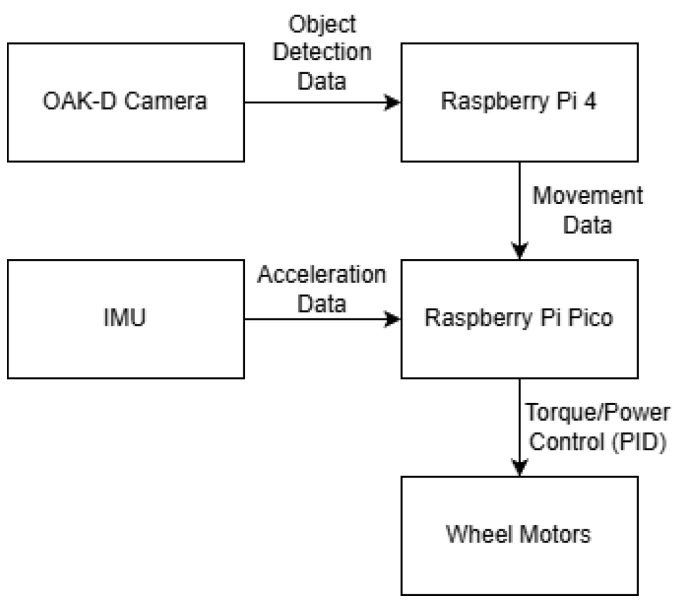

Fig. 2. System block diagram. The Pi 4 hosts ROS2 and the joystick teleoperation node; the Pico hosts the real-time balance and velocity loops. The OAK-D Pro and a vision-pipeline placeholder are shown for completeness; no vision software is integrated in the delivered system.

Every custom-fabricated part on the robot was designed in

CAD by the author and produced in-house at the Northeastern FlexLab maker space. The chassis is a hybrid build that uses three distinct fabrication processes and three materials, each chosen for the specific structural role it plays. Three or more laser-cut acrylic decks form the flat structural levels of the chassis; PLA-printed brackets and mounts attach the motors, battery, camera, and supports to those decks; and PETG-printed bushings couple the motor shafts to the wheels at the single highest-load mechanical interface on the robot. Twelve custom parts were designed and produced for the platform across these three categories. CAD files for all custom parts are included in the project repository.

Laser-cut acrylic decks. The chassis is built around three horizontal acrylic decks stacked vertically and joined by metal standoffs. The decks carry the electronics (motor driver, Pico, wiring, battery) on the lower levels and the cargo platform on the upper level. Acrylic was selected for the decks because the deck geometry is essentially two-dimensional: large flat panels with regular hole patterns for component mounting and standoff connections. Laser cutting produces this geometry in a single pass with tight tolerances and smooth edges, and acrylic is dimensionally stable enough that the standoff hole patterns line up reliably across all decks for clean stacking.

PLA-printed brackets and mounts. The bracketry that attaches components to the acrylic decks is 3D-printed in PLA. This includes the FIT0186 motor mounts, the OAK-D Pro camera mount, the Anker battery holder for the logic-side power pack, the 12 V motor battery holder, and the structural supports that brace the deck stack against bending. PLA was the default printer feedstock available in the FlexLab and was the right choice for these parts: the brackets carry static or low-cycle loads (holding components in place against gravity and occasional handling) rather t han the repeated high-cycle loads at the wheel-motor interface, and PLA's stiffness and dimensional accuracy serve the mounting role well.

PETG-printed wheel bushings. The single highest mechanical demand on the robot is the wheel hub bushing, which couples each motor's flatted D-shaft to the round bore of the 8-inch polyurethane wheel. Every newton-meter of motor torque is transmitted through this part, and it sees repeated impact loads from the wheel-ground contact during balancing, driving, and disturbance recovery. The bushings (`Bush_v2.stl`) were therefore printed in PETG rather than PLA, because PETG offers materially better toughness and creep resistance under cyclic and impact loading than PLA. PETG was scarce in the FlexLab inventory and was obtained for these parts by request to the lab staff; reserving it for the bushings, where its mechanical advantage actually matters, was a deliberate allocation given the limited supply. The bushings were also the most-iterated mechanical part on the robot. Several print revisions were required to converge on a fit that gripped the motor's flatted shaft tightly enough not to slip under torque, while remaining loose enough to install and remove without damaging the wheel.

Design rationale and fabrication constraints. The hybrid construction reflects a deliberate match between fabrication processes and the loads each part carries: laser-cut acrylic for flat structural geometry, 3D-printed PLA for shaped low-load bracketry, and 3D-printed PETG only at the single mechanical interface where its toughness was needed. The chassis dimensions were further constrained by the working envelopes of the available equipment in the FlexLab: the laser cutter's bed sets an upper bound on deck dimensions, and the 3D printer's build volume sets an upper bound on the size of any single printed part. The latter constraint forced several mechanical functions to be split across multiple printed parts rather than designed as unified brackets. Producing all custom parts in-house from CAD also kept iteration cost low.

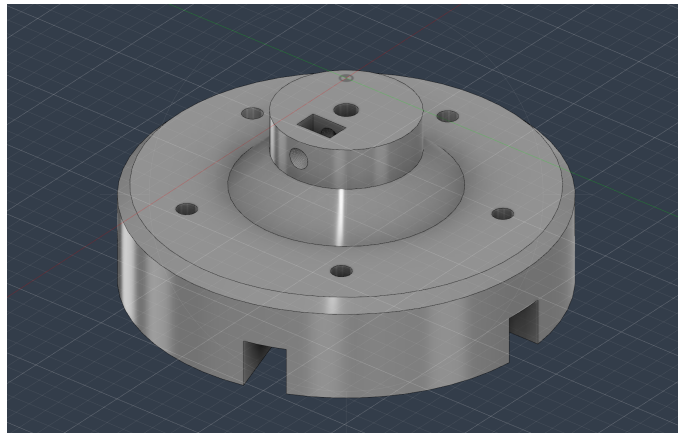

Fig. 3. CAD render of the PETG-printed wheel bushing (`Bush_v2.stl`), which couples the FIT0186 motor's flatted D-shaft to the 8-inch polyurethane wheel. This is the single highest-load custom part on the robot.

C. Software Architecture

The CAMS-Bot software stack is split across the two compute boards described in Section 3.2, with the split chosen to isolate the deterministic real-time balance loop from the non-deterministic timing of Linux, ROS2, and any future perception pipeline. Figure 4 shows the block diagram of the delivered stack. The Raspberry Pi 4 hosts the high-level layer (Ubuntu, ROS2 Jazzy in Docker, the joystick teleoperation node, and the serial bridge to the Pico). The Raspberry Pi Pico hosts the low-level layer (the MicroPython balance controller, encoder decoding, and the L298N motor driver interface). The two boards communicate over a hardware UART link with a small ASCII protocol described below.

High-level layer (Raspberry Pi 4). The Pi 4 runs Ubuntu with ROS2 Jazzy hosted inside a Docker container. Containerizing the ROS2 environment isolates the project's dependencies from the host system and makes the high-level stack reproducible: the same Docker image can be rebuilt on a fresh Pi 4 image with no manual ROS2 installation. Two ROS2 nodes are deployed in the container. The *joystick node* reads input from an Xbox controller connected over USB using `pygame` and publishes the controller state to the standard ROS2 `/joy` topic. The *teleoperation node* subscribes to `/joy`, maps the controller's left-stick Y axis to a normalized

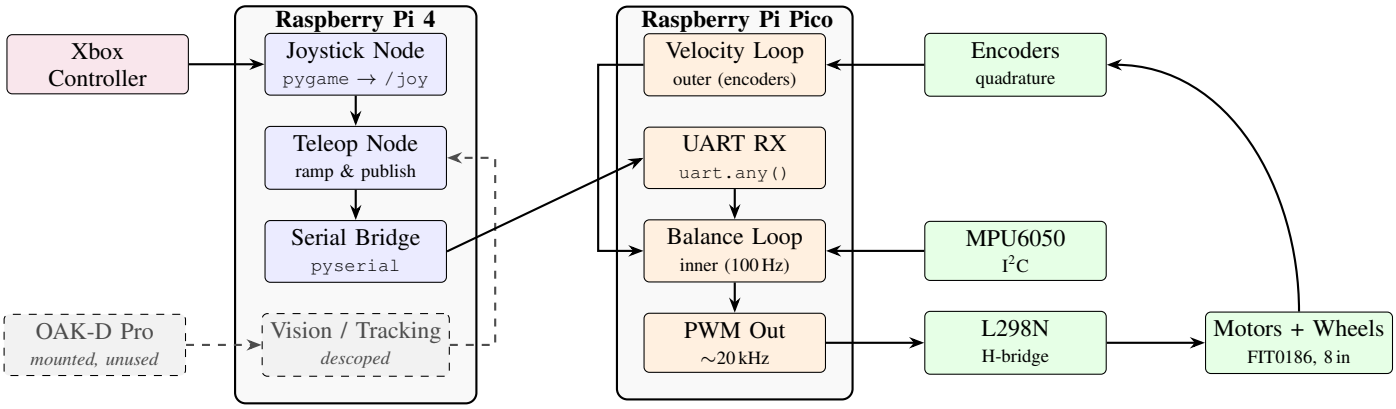

Fig. 4. Software architecture and signal flow. The Raspberry Pi 4 (left, blue) runs ROS2 Jazzy in Docker with a joystick node, a teleoperation node, and a serial bridge. The Raspberry Pi Pico (center, orange) runs the cascaded PID controller in MicroPython: an outer velocity loop biases the inner balance loop's tilt setpoint, and the inner loop drives the L298N motor driver at ∼20 kHz PWM. Hardware peripherals are shown in green. The OAK-D Pro camera and vision/tracking node (dashed) are physically present but were descoped from the delivered system.

drive command and the right-stick X axis to a normalized turn command, ramps the commanded values at 20 Hz inside a timer callback to avoid step changes that would shock the balance loop, and writes the resulting commands to the Pico over a serial port using `pyserial`.

The serial port is exposed as a stable symlink `/dev/ttyPICO` via a udev rule keyed on the Pico's USB vendor ID (`2e8a`). This rule guarantees that the device path does not change across reboots or USB re-enumerations, which removes a class of bugs where the teleoperation node would attach to the wrong device after a hot-plug.

Low-level layer (Raspberry Pi Pico). The Pico runs the real-time balance controller in MicroPython at a 100 Hz inner-loop rate. On each loop iteration the Pico (i) reads the MPU6050 IMU over I^2C, (ii) estimates the chassis tilt angle from the accelerometer and gyroscope using a complementary filter, (iii) reads the wheel encoders to estimate wheel velocity, (iv) runs the cascaded PID controller described in the next paragraph, and (v) writes PWM commands to the L298N motor driver. The PWM frequency to the L298N is approximately 20 kHz, which sits above the audible range and produces smoother current delivery to the gearmotor inductance than lower switching frequencies.

Control: cascaded PID architecture. Balance and motion are produced by a cascaded two-loop controller. The inner *balance loop* runs every control tick and regulates chassis tilt: it produces a motor PWM command from the tilt error (measured tilt minus commanded tilt setpoint) and the tilt rate, through a PID controller with proportional, derivative, and small integral terms. The derivative term is filtered through an exponential moving average to suppress gyroscope noise, and the integral term is clamped to prevent windup during disturbances. The outer *velocity loop* runs at a lower rate, divided down from the inner-loop rate by an integer factor (`VEL_LOOP_DIV`). It compares the wheel-velocity estimate from the encoders against the commanded drive value from the high-level layer and slowly biases the inner loop's tilt setpoint accordingly. This is the mechanism by which the robot translates a "drive forward" command into actual motion: the velocity loop deliberately leans the chassis in the desired direction, and the balance loop drives the wheels to keep up. A separate turn channel adds and subtracts a differential PWM bias to the two motor commands to produce yaw motion. The specific gain values and tuning history are reported in Section 4.

User interface. The user interface is a wired Xbox controller connected to the Pi 4 over USB. The left stick Y axis controls forward and reverse drive; the right stick X axis controls turning. Drive and turn inputs are normalized to the range $[-1, 1]$ on the Pi 4 side and transmitted to the Pico in the ASCII protocol described next. A graphical or voice interface was not in scope.

Communication: Pi 4 to Pico ASCII protocol. The two boards communicate over a hardware UART link with a deliberately simple line-oriented ASCII protocol. Drive and turn commands are sent as `D<float>,T<float>\n`, where `D` is the normalized drive value and `T` is the normalized turn value, both in $[-1, 1]$. An immediate stop is sent as `S\n`. The Pico's UART receive uses `uart.any()` to check for available bytes before reading, which gives a non-blocking read pattern compatible with the 100 Hz control loop. An earlier design used USB CDC serial on the Pico's USB device interface; this was abandoned after persistent blocking-read failures, in which `readline()` would block on incoming newlines, `read(n)` would block on byte counts, and even single-byte buffered reads were sufficient to break the control loop's timing budget. Switching to hardware UART with `uart.any()`-gated reads resolved the problem cleanly.

Perception (descoped). The system architecture diagram shows a dashed perception module corresponding to the OAK-D Pro depth camera and the originally planned DepthAI and YOLO-based person-tracking pipeline. As discussed in Section 3.1, this subsystem was not delivered. The camera is physically mounted on the chassis and connected over USB

to the Pi 4, but no vision software is integrated, no perception nodes are present in the ROS2 graph, and no autonomous mode runs on the Pi 4. The architecture is built so that adding the perception module would not require changes to the high-level node graph beyond inserting the node itself: the teleoperation node's command interface (the same drive and turn channel forwarded over UART) is the natural insertion point for autonomous commands.

Planning (descoped). No planner runs on the delivered system. The proposed autonomous-following behavior would have used the perception module's person-position estimate to compute drive and turn commands directly, without a separate planning layer; that simple control law was the planning equivalent in the original design and would attach at the same insertion point as the perception module above. Free-space planning, obstacle-avoidance planning, and map-based navigation were never in scope.

Safety. The delivered safety mechanism is a hardware kill switch that cuts power to the motors when pressed, providing the operator with an immediate physical override. As discussed in Section 3.1, this satisfies the operator-in- the-loop case but does not satisfy the proposal's automatic-tilt-cutoff requirement, since fall-class tilt events complete in well under a second. An automatic tilt-threshold cutoff in the Pico balance loop is identified as future work in Section 7.

Changes from the proposal. Five elements of the software architecture changed between the proposal and the delivered system. (1) The motor driver was changed from the TB6612FNG specified in the proposal to the L298N, after the TB6612FNG produced reliability problems under the load conditions of the 12 V gearmotors. (2) The Pico balance controller is implemented in MicroPython rather than the C++ originally specified, because MicroPython provided a faster iteration loop on the Pico for the tuning-heavy development style the project required. (3) The inner balance loop runs at 100 Hz rather than the 200 Hz specified in the proposal; 100 Hz proved sufficient for stable balance and was the rate at which the IMU read, encoder decode, and PID computation comfortably fit within the loop period. (4) The Pi 4 to Pico link is hardware UART rather than USB CDC, after the USB CDC implementation was found to introduce blocking-read behavior incompatible with the control loop's timing budget. (5) The autonomous-following capability based on the OAK-D Pro and DepthAI was descoped, as discussed at length elsewhere in this report. The first four changes are technical refinements that produced a more reliable delivered system; the fifth is a scope reduction that resulted from the project partner's withdrawal from the course.

IV. Experimental Evaluation

A. System Tasks

The CAMS-Bot proposal defined the primary assistive task as autonomously following a user while carrying a load in a residential environment, decomposed into five subtasks: person detection and acquisition, person following, load carrying under balance, obstacle response, and manual override. As discussed in Section 3, four of these five subtasks (all but load carrying and manual override) depend on the OAK-D Pro vision and DepthAI person-tracking pipeline, which was descoped from the delivered system. Evaluating the delivered system against the original five-subtask decomposition would consist mostly of reporting "not delivered" for vision-dependent rows, which is uninformative.

This section instead evaluates the delivered system against the subset of tasks that the as-built hardware and software actually perform. Three tasks are evaluated: dynamic balance, teleoperated drive and turn, and load carrying under balance. These three tasks together exercise the full delivered stack, from the Xbox controller through the Pi 4 teleoperation node, the UART link, the Pico cascaded PID controller, and the L298N motor driver to the wheels and the chassis. They are also the three tasks against which the headline assistive specification of the project (the 5 kg payload requirement) is meaningful: a balanced platform that accepts manual drive commands and holds 5 kg of cargo is what the delivered system is, and is the closest defensible approximation of the proposal's load-carrying-under-balance subtask.

Task 1: Dynamic balance. The robot must maintain its upright pose against the natural disturbances of the chassis and surface (small irregularities in the floor, residual asymmetries between the two motor channels, wiring tug, and so on) for an extended period without operator command. This task tests the inner balance loop in isolation and is the first task the controller had to perform during PID tuning.

Task 2: Teleoperated drive and turn. The robot must respond to drive and turn commands issued from the Xbox controller through the ROS2 teleoperation node, translating those commands into forward and reverse motion and yaw rotation while continuing to balance. This task exercises both the high-level stack (joystick node, teleoperation node, UART serial bridge) and the cascaded PID controller (the outer velocity loop that biases the inner loop's setpoint to produce drive motion, and the differential-PWM mechanism that produces yaw motion).

Task 3: Static load-bearing and loaded locomotion. The robot must perform Tasks 1 and 2 while carrying a payload on the cargo platform, up to the project's headline 5 kg target. This task is the assistive specification of the project: a balance platform that cannot carry meaningful cargo is not useful for the assistive task in Section 1. Task 3 is decomposed into two distinct subcases, because the delivered system's behavior differs sharply between them. The static subcase (Task 3a) requires the robot to hold the upright pose with 5 kg of cargo on the platform, without commanded motion. The dynamic subcase (Task 3b) requires the robot to perform the drive and turn behaviors of Task 2 while loaded with 5 kg. As detailed in Section 4.4, the delivered system passes Task 3a and fails Task 3b, and the failure mechanism is geometric rather than control-related: the cargo platform sits above the wheel axle, and on the current 8-inch wheels the lean required to initiate motion under the velocity loop swings the cargo's center of mass past the contact patch before the controller can translate

the chassis forward.

The vision-dependent tasks from the proposal (person detection and acquisition, person following, and obstacle response) are not evaluated. The proposal's metrics for these tasks (following success rate, following distance error, and obstacle stop distance) are reported as not-applicable in Section 4.4 rather than reported with degenerate values, since the conditions of measurement (an autonomous mode, a tracked user, and a perception-driven obstacle response) are not present in the delivered system.

B. Metrics

The proposal defined five quantitative metrics for system evaluation: following success rate, following distance error, balance stability under load, obstacle stop distance, and override response time. Three of the five (following success rate, following distance error, and obstacle stop distance) measure the autonomous-following behaviors that depend on the descoped vision pipeline and are not applicable to the delivered system. Section 4.4 reports those rows as "not applicable" rather than zero, since their conditions of measurement (an autonomous mode, a tracked user, and a perception-driven obstacle response) are not present in the delivered system.

The remaining two proposal metrics, balance stability under load and override response time, apply to the delivered system but in modified form. The proposal's balance stability metric specified zero falls per trial across three payload levels (0 kg, 2.5 kg, and 5 kg), with each trial consisting of a 10 m walked path. The delivered system was tested at the binary endpoints of this payload range (unloaded driving and static 5 kg balance) rather than across the swept payload sweep the proposal specified, for the reasons discussed in Section 4.1 and Section 4.5. The override response metric (time from controller input to motor stop) applies directly to the delivered system, since both the hardware kill switch and the `S` serial stop command produce an immediate motor cutoff observable to the operator.

The metrics applied in this evaluation are organized by the three tasks defined in Section 4.1.

Task 1: Dynamic balance.

- *Static stability outcome.* Whether the unloaded chassis maintains upright pose without operator command for the duration of a test trial. Reported as a pass/fail outcome from video evidence.
- *Surface dependence.* Qualitative comparison of stability on residential carpet versus smooth flooring (hardwood and tile). Reported as observed behavior.
- *Tuning convergence.* The PID gain values that produced acceptable balance behavior, including the proportional gain K_P, the integral gain K_I, the derivative gain K_D, the tilt setpoint, the deadband, and the minimum effective PWM. Reported as the final tuned configuration.

Task 2: Teleoperated drive and turn.

- *Command responsiveness.* The qualitative responsiveness of the chassis to drive and turn commands from the Xbox controller, including any observed latency or non-linearity between input and motion. Reported from video evidence.
- *Override response.* The behavior of the kill switch and the `S` serial stop command. Reported as observed cutoff behavior.

Task 3a: Static load-bearing.

- *Static load capacity.* The maximum payload the chassis and balance controller hold in static upright pose without falling. Reported as the binary outcome of the static 5 kg test (pass/fail).

Task 3b: Loaded locomotion.

- *Dynamic load capacity.* Whether the chassis can perform Task 2 (drive and turn) while loaded with 5 kg. Reported as the binary outcome of the loaded-driving test (pass/fail).

A note on quantitative versus qualitative reporting. The metrics defined here are largely qualitative or binary, rather than the continuous quantitative measures the proposal specified. This reflects two practical realities of the delivered evaluation. First, the descoped autonomous mode removes the only context in which several of the proposal's continuous metrics (following distance error, obstacle stop distance, following success rate over many trials) would have been meaningful. Second, the project's instrumentation and timeline did not include logged tilt-angle traces, encoder-velocity traces, or other continuous signals that would have permitted RMS-error or bandwidth reporting of the kind that fully quantifies a balance controller. The value of automated logging for future builds of this kind of system is revisited in Section 5 (Lessons Learned).

C. Experimental Procedure

The experimental work on CAMS-Bot was hand-tuning of the cascaded PID balance controller, followed by integration testing of the teleoperation stack and the load test. This section documents the procedure in enough detail for reproducibility on a similar build.

Setup conditions. Tuning was conducted indoors on residential carpet, which is the surface on which the controller behaves most reliably (Section 4.4). The robot was powered from the split power architecture described in Section 3.2 (12 V battery for the motors, USB battery pack for the Pi 4 and Pico). The Xbox controller was paired to the Pi 4 by USB. The MicroPython balance controller was edited and flashed using the MicroPico extension in VSCode, and a serial monitor on the same workstation provided real-time access to the Pico's print output during tuning. A toggle bound to the keypress `p` over the serial monitor enabled the Pico to print the tilt angle and gyroscope rate at 10 Hz while balancing, which was the primary diagnostic signal during gain selection.

Pre-tuning checks. Before any controller gain was set, three pre-conditions were verified on each session, because each had been the cause of an earlier debugging session that wasted time and is documented here so that future builds do not repeat them. (i) The MPU6050 boot sequence was given a

2 s startup delay in software before the first I^2C read, since shorter delays produced intermittent boot failures of the IMU. (ii) The IMU sign convention was verified empirically by tilting the chassis forward by hand and confirming the sign of the reported angle (forward tilt produces a negative value in the firmware's convention), since a sign error in the IMU reading inverts the controller and produces an immediate fall. (iii) A motor-deadband feedforward was added to compensate for the gearmotors' nonzero electrical start threshold. The DFRobot FIT0186 gearmotors do not begin to rotate below approximately 1 V at the motor terminals, so PWM commands below this threshold produce no motion regardless of polarity, and the closed loop is effectively open in that range. Rather than apply a simple floor (which an earlier version of the firmware tried, and which produced visible oscillation at the deadband boundary because of the discontinuous PWM step), the delivered firmware holds the motor fully off below an absolute PID output of `MIN_CMD` = 100 and linearly ramps a feedforward offset from zero up to `MOTOR_OFFSET` = 30000 between `MIN_CMD` and `TRANSITION` = 2800, with sign preserved. This produces a smooth zero-crossing at the motor's start threshold and ensures that any controller correction the loop wants to issue is actually executed.

Tuning sequence. The PID tuning proceeded in the following order, which is the order recommended by the Instructables reference build [16] and is the order that converged in practice:

1) *K_P alone (pure-P loop).* With $K_I = K_D = 0$, K_P was increased from a small value upward until the controller produced a visible corrective response when the chassis was deflected by hand. The diagnostic for "K_P is in the right neighborhood" is that the chassis fights small disturbances and tries to return to vertical, but overshoots in the form of low-frequency oscillation around the upright pose. A trial with this configuration ends when the chassis falls, when the operator stops the trial via the kill switch, or when the chassis settles into oscillation. This step also doubled as a sanity check on the IMU sign convention and the motor wiring polarity: a controller that drives the chassis *further* from vertical when deflected is sign-inverted somewhere, and was caught at this step rather than later.
2) *Add K_D.* With K_P fixed near its single-gain limit, K_D was increased from zero until the low-frequency oscillation from Step 1 was visibly damped. The diagnostic for "K_D is in the right neighborhood" is that small hand-applied disturbances are caught early in the tilt and corrected smoothly, rather than letting the tilt grow into the regime where K_P produces an aggressive correction. The derivative term was filtered through an exponential moving average on the tilt rate, because raw gyroscope noise without filtering produced high-frequency motor chatter at large K_D values. A trial at this step ends on a fall, a kill-switch stop, or an extended period of quiet upright balance.
3) *Adjust the tilt setpoint.* A persistent slow drift in one direction during otherwise quiet balance is a sign that the commanded tilt setpoint does not match the chassis's physical balance point. The setpoint was adjusted by small increments (on the order of 0.1 deg) until the drift disappeared. The setpoint was tuned before K_I rather than relying on K_I to correct the steady-state offset, because integral action on a wrong setpoint produces windup rather than convergence.
4) *Add K_I (small).* With the setpoint correctly placed, K_I was increased from zero in small steps. The integral term was clamped to a finite range to prevent windup during disturbances, and was zeroed when the controller entered the deadband (defined below). For most of the development cycle K_I remained at zero, since the correctly-placed setpoint plus the P and D terms produced acceptable steady-state behavior; the integral gain was used only when a small persistent offset reappeared after mechanical changes to the chassis (the addition of the camera mount, for example, shifted the balance point by enough to require either a setpoint retune or a small integral correction).
5) *Engage the velocity loop.* With the inner balance loop stable, the outer velocity loop was engaged. The velocity loop reads wheel velocity from the encoders, compares it against the commanded drive value from the Pi 4, and biases the inner loop's tilt setpoint accordingly. The velocity-loop divisor (`VEL_LOOP_DIV`) was set so that the velocity loop runs at one-tenth the inner-loop rate, since a velocity loop that runs as fast as the balance loop produces inner-outer interaction (the velocity loop reacts to the balance loop's transients) rather than treating the inner loop as a settled plant. Velocity-loop gains were small relative to the balance-loop gains, since their job is to bias rather than dominate the setpoint.

Deadband behavior. A small deadband was applied around the tilt setpoint inside which the controller output was zero, to prevent the controller from chattering the motors against the natural noise floor of the IMU and the encoders while the chassis was already approximately upright. A subtle bug surfaced during tuning and is documented here: the deadband must *not* reset the previous-error and filtered-derivative state to zero on entry. Resetting those values caused a derivative spike on each deadband re-entry, which produced visible motor twitches at the deadband boundary and degraded overall balance. The corrected behavior preserves `prev_error` and `filtered_d` across deadband transitions; the deadband only zeroes the output, not the controller's internal state.

Trial structure. A balance trial began with the operator holding the chassis upright by hand, releasing it once the controller appeared to engage, and observing the chassis behavior until one of three terminating conditions was reached:

- *Fall:* the chassis tilted past the operator's recoverable angle (approximately 30° from vertical) and was caught by hand or by the floor. The kill switch was used in this case to cut motor power before contact with the floor

when possible.

- *Operator stop:* the operator pressed the kill switch in response to oscillation, drift, or any unsafe behavior, ending the trial without a fall.
- *Success:* the chassis maintained upright pose, with motion confined to small natural disturbances of the surface and the wiring, for a duration sufficient to demonstrate stable balance.

A new trial began either by editing the controller gains in the firmware and re-flashing the Pico (for tuning iterations), or by simply restarting the controller with the same gains (for repeat trials of a converged configuration).

Teleoperation and load tests. Once the balance controller was tuned, the Xbox controller and the ROS2 teleoperation node were brought up on the Pi 4, and Task 2 (drive and turn) was tested by issuing drive and turn commands while the chassis balanced. The load tests for Task 3 used a known mass placed on the cargo platform: a successful Task 3a trial held the chassis upright with the 5 kg payload for the duration of a video-recorded clip; a Task 3b trial issued small drive commands under the loaded condition and was terminated by the resulting fall (Section 4.4).

Reproducibility. The MicroPython firmware, the ROS2 nodes, the udev rule for `/dev/ttyPICO`, and the CAD files for all custom hardware are included in the project repository. With access to the same hardware bill of materials, a reasonable-fidelity reproduction of the delivered system should be achievable in a single semester. The hard-won lessons that saved the most debugging time, listed in priority order, are: (i) set `MIN_PWM` above the motor's electrical start threshold; (ii) delay the first IMU read by 2 s after Pico boot; (iii) filter the derivative term and clamp the integral term; (iv) preserve controller state across deadband transitions; and (v) tune the setpoint before introducing K_I. These items are revisited as recommendations for future teams in Section 5.

D. Results

Table II reports the outcomes of the metrics defined in Section 4.2, organized by the three tasks from Section 4.1. Quantitative values are reported where they were measured; qualitative or binary outcomes are reported where the metric was qualitative or binary by design. The remainder of the section expands on the entries that require more than a one-cell summary.

TABLE II
RESULTS BY TASK AND METRIC

Metric	Outcome
Task 1: Dynamic balance	
Static stability outcome	Pass on carpet
Surface dependence	Reliable on carpet; degraded on hardwood/tile
Tuning convergence	Final gains in Table III
Task 2: Teleoperated drive and turn	
Command responsiveness	Pass; smooth response to drive and turn
Override response	Pass; immediate motor cutoff on kill switch and `S` stop
Task 3a: Static load-bearing	
Static load capacity	Pass at 5 kg
Task 3b: Loaded locomotion	
Dynamic load capacity	Fail at 5 kg (geometric tipping; see below)
Descoped (vision-dependent)	
Following success rate	Not applicable
Following distance error	Not applicable
Obstacle stop distance	Not applicable

Tuning convergence (Task 1). The PID tuning sequence described in Section 4.3 converged to the gain set reported in Table III. These are the values that were live in the firmware during the video-documented balance and load tests, and constitute the controller configuration that produced the Task 1 and Task 3a passes.

Surface dependence (Task 1). The controller is tuned for, and behaves most reliably on, residential carpet. On smooth flooring such as hardwood and tile the same controller produces visibly less stable behavior: small disturbances that the carpet case rejects without operator intervention can grow on smooth flooring into oscillations large enough to require a kill-switch stop. Two contributing causes are likely. First, carpet adds rolling resistance that passively damps small velocity excursions before the velocity loop reacts to them, while smooth flooring offers no such damping. Second, the 8-inch polyurethane wheels selected for the project are intended for smooth warehouse-style floors at much higher loads than this robot applies; their durometer and tread profile produce a small, hard contact patch on smooth flooring that gives the controller less margin than a softer or larger wheel would. The combination of the as-built wheels and the controller's lack of any surface-adaptive behavior produces the carpet-versus-smooth-floor asymmetry observed during testing.

Command responsiveness and override (Task 2). The teleoperated drive and turn behaviors performed as designed. Drive commands from the Xbox controller's left-stick Y axis produce smooth forward and reverse motion on the chassis, with the velocity loop biasing the inner loop's tilt setpoint to lean the chassis in the commanded direction. Turn commands from the right-stick X axis produce yaw rotation through the differential-PWM mechanism. There is no observable latency between controller input and chassis response beyond what the 20 Hz teleoperation publish rate and the 100 Hz inner loop together permit. The override response is immediate by construction: the hardware kill switch cuts power to the motors at the electrical level, and the `S` serial stop command is executed by the Pico on the next inner-loop iteration after the byte arrives over UART, both of which produce a motor cutoff faster than an operator can perceive a delay.

Static load-bearing at 5 kg (Task 3a). The chassis, decks, motor mounts, wheel bushings, and balance controller hold the bot upright with 5 kg of cargo on the platform. The controller's

TABLE III
FINAL TUNED CONTROLLER CONFIGURATION

Parameter	Value	Notes
Inner balance loop (PID)		
K_P	1000.0	Proportional gain
K_I	0.0	Disabled; infrastructure clamped
K_D	700.0	EMA-filtered (D_FILTER_ALPHA=0.3)
Tilt setpoint	$-1.0°$	Empirical balance point
Deadband	$0.3°$	State-preserving
Integral zone	$2.0°$	Engages K_I inside this band
Integral clamp	± 8.0	Anti-windup limit
Outer velocity loop (PID)		
VEL_KP	0.002	Velocity proportional
VEL_KI	0.0006	Velocity integral
VEL_KD	0.0	Disabled
VEL_LOOP_DIV	10	Outer/inner rate ratio
VEL_INTEGRAL_CLAMP	± 1.5	Anti-windup limit
VEL_MAX_OFFSET	$\pm 2.0°$	Max setpoint bias
VEL_D_FILTER	0.08	Derivative low-pass
Drive input shaping		
MAX_DRIVE_ANGLE	$1.5°$	Lean at full stick
VEL_DRIVE_SCALE	5500	Trigger $\rightarrow$ velocity target
TURN_SCALE	10000	Stick $\rightarrow$ turn differential
VEL_RAMP_RATE	4000	Velocity target slew
DRIVE_RAMP_RATE	8.0	Drive target slew
TRIGGER_EDGE	0.2	Stick deadzone
Stiction-overcome kick		
KICK_PWM	12000	Initial pulse from rest
KICK_DECAY	0.88	Per-tick decay factor
KICK_MIN	200	Floor before kick disengages
Output conditioning		
MAX_OUTPUT	65535	16-bit Pico PWM full scale
MIN_CMD	100	Below this $\lvert u \rvert$, motor fully off
MOTOR_OFFSET	30000	Deadband feedforward (full)
TRANSITION	2800	PID output where offset is fully active
Output filter α	0.3	EMA on commanded output
IMU fusion and rates		
Complementary α	0.98	Gyro vs. accelerometer weight
Inner-loop rate	100 Hz	MicroPython on Pico
PWM frequency	$\sim$20 kHz	L298N drive

existing gains (Table III), tuned for the unloaded chassis, are sufficient to maintain static balance under the increased mass and the raised center of gravity. This passes the project's headline assistive specification on the static portion of the load-carrying task.

Loaded locomotion failure (Task 3b). The 5 kg loaded locomotion case fails. The mechanism, summarized in Section 4.1 and analyzed in Section 4.5, is geometric: the cargo platform sits above the wheel axle, and on the current 8-inch wheels even a small commanded lean swings the cargo's center of mass past the wheel's contact patch before the velocity loop can deliver enough wheel acceleration to translate the chassis forward. Once the cargo center of mass crosses the contact patch, gravity's moment arm exceeds what the motors can counter and the chassis tips. This failure is not addressable by gain tuning, since the lean that produces the failure is the same lean the velocity loop commands to initiate motion. Section 7 (Future Work) identifies larger-diameter wheels (in the range of 24–29 inches) and a chassis redesign that places the cargo platform closer to or below the wheel axle as the architectural changes that would address the failure.

Tuning headroom. The gain set in Table III produced acceptable balance behavior across the test trials but does not represent a maximally tight tuning. Two further refinements are visible from the test behavior. First, the controller is non-adaptive: the same gain set is applied across all surface conditions and all loaded conditions, and the surface-dependence asymmetry above and the loaded-locomotion failure each arise in part from the absence of a plant-conditioned gain schedule. A gain-scheduled controller, or an autotuning mechanism that adjusts gains in response to measured plant response, would address both. Second, the inner-loop gains were tuned by hand against visible chassis behavior rather than against a logged tilt trace; a logging-driven retune would tighten the gains by allowing overshoot, settling time, and steady-state error to be measured quantitatively rather than estimated visually. These refinements are identified as future work in Section 7.

E. Analysis and Discussion

Performance against requirements. The delivered system meets the proposal's headline static specification (carrying 5 kg) and meets the operator-control and override requirements, but falls short of the dynamic 5 kg specification and of the autonomous-following functional set. The proposal's payload requirement was articulated as "carry payloads up to 5 kg while maintaining balance," which on a literal reading the static load test (Section 4.4, Task 3a) satisfies. On a more demanding reading, in which "carry" implies locomotion, the dynamic load test (Task 3b) does not satisfy the requirement. The honest verdict is partial: the structural and balance work that the static 5 kg case demands is delivered, and the architectural work the dynamic 5 kg case demands is not. The autonomous-following functional requirements (following at 0.5–1.5 m/s, distance keeping, obstacle response, untethered tracking) were descoped with the withdrawal of the project partner responsible for the vision pipeline, and the safety requirements that depend on those autonomous behaviors (stop on loss of detection, autonomous-mode speed cap) are descoped with them. The remaining safety requirement (motor cutoff on excessive tilt) was met partially: an operator-initiated kill switch is delivered, where an automatic in-loop tilt cutoff was specified.

Failure mode analysis: loaded locomotion (Task 3b). The dynamic 5 kg failure is the most significant performance gap and deserves a precise analysis. The mechanism is geometric, not control- related, and the distinction matters for how the failure is addressed. The cargo platform sits above the wheel axle by the height of the intermediate chassis decks. The

velocity loop produces motion by biasing the inner balance loop's tilt setpoint, deliberately leaning the chassis in the commanded direction so that the inner loop will drive the wheels forward to chase the lean. Under load, the cargo mounted at the top of the chassis swings forward in an arc around the wheel axle as the chassis leans. The horizontal displacement of the cargo's center of mass for a given lean angle scales with the height of the cargo above the axle. With 5 kg of cargo on the current chassis, the lean angle that the velocity loop must command to initiate even a small commanded velocity is large enough that the cargo's center of mass crosses outside the wheel's contact patch before the wheel can translate the chassis forward. Once the cargo center of mass exits the contact patch, gravity acts with a moment arm the motors cannot counter, and the chassis tips before the controller can recover.

This failure cannot be addressed by gain tuning, because the lean that produces the failure is the same lean the controller commands to initiate motion. Tightening the gains would let the controller fight small leans harder, but the architecture requires the chassis to *lean to move*, and any commanded motion under a high cargo crosses the geometric tipping margin. The failure is therefore architectural and admits two architectural fixes (developed in Section 7): scale up the wheel diameter so that the geometric tipping envelope is larger and the lean angle needed to produce a given translational acceleration is smaller, or lower the cargo platform toward or below the wheel axle so that the moment arm of the loaded center of mass is reduced. Either fix changes the geometry of the platform; both together would address the failure most robustly.

Failure mode analysis: surface dependence (Task 1). The carpet-versus-smooth-floor asymmetry observed in Section 4.4 is a secondary failure mode of the controller in its current form. Two contributing causes were identified. The first is the absence of a plant-adaptive control mechanism: the gain set tuned on carpet is applied unchanged on hardwood and tile, where the rolling-resistance damping that the carpet contributes is absent and the controller operates with less effective damping margin. The second is the wheel selection. The 8-inch polyurethane wheels were specified for traction on smooth warehouse-style flooring at higher loads than this robot applies, and on residential smooth flooring under this robot's load they produce a small hard contact patch with little compliance. A softer or pneumatically-tired wheel would absorb small surface irregularities and provide some natural damping that the rigid polyurethane does not.

Comparison to related work. The CAMS-Bot result is consistent with the controls literature reviewed in Section 2. The cascaded PID architecture adopted from Petcu and Andreescu [14] and the Instructables reference build [16] produced stable balance behavior on the delivered system, confirming that PID is a sufficient control strategy for the inner balance loop on a small two-wheeled platform when implemented carefully (correct sign convention, derivative filtering, integral clamping, and a properly placed setpoint). The platform-level result, however, diverges from the cargo-and-following platforms in Section 2 in a way that highlights the gap CAMS-Bot was attempting to close. The Foxtech FOLO-200 commercial follower [10] solves the loaded-locomotion problem by being mechanically passive: it uses four wheels on a stable base and never has to lean to move, so the geometric tipping problem CAMS-Bot encountered does not arise. The inverted-pendulum platforms in the academic literature [14], [15] typically demonstrate balance without payload, and where payload is included it is mounted close to the wheel axle rather than on a raised cargo platform, so the geometric tipping mode at large loaded leans is rarely encountered. The CAMS-Bot delivered system sits between these regimes: it adopts the residentially- scaled two-wheel form factor of the academic platforms but attempts to carry the residentially-meaningful payload of the commercial follower class, and the geometric mismatch between those two ambitions on the current hardware is the dominant lesson of the evaluation.

Implications for a more robust build. The analyses above imply a specific set of architectural changes for a follow-on build that would address the dominant failure modes. (i) The wheels should be larger in diameter (in the range of 24–29 inches) and should be pneumatic rather than rigid polyurethane, both to extend the geometric tipping envelope under load and to provide passive compliance on smooth flooring. (ii) The chassis should be metal rather than the laser-cut acrylic and 3D-printed PETG and PLA of the current build, since metal admits a stiffer structure at the larger scale that the bigger wheels imply, and admits a solid drive axle through the chassis that would replace the current motor-mounted-to-deck arrangement. (iii) A solid axle through the chassis would let the cargo platform be mounted closer to or below the axle rather than only above it, which is the second of the two geometric fixes for the loaded-locomotion failure. (iv) The vision and tracking subsystem, descoped from the delivered build, completes the autonomous functional set and is the natural software addition once the platform is hardware-robust enough to carry a camera through the residential environment without falling. These changes are developed as concrete future-work items in Section 7.

Encoder-based velocity feedback was instrumental. A specific positive finding of the evaluation is that the outer velocity loop, driven by quadrature-encoder feedback from the two gearmotors, was essential rather than optional for the controllable behavior the delivered system achieves. A balance-only configuration with no velocity feedback (an inner-loop-only PID against tilt) holds the chassis upright in static balance, but cannot be commanded to drive in any controlled way and tends to drift on the floor under small asymmetries between the two motor channels. Adding the outer velocity loop, in which the encoder-derived wheel velocity is compared against the commanded drive value and used to bias the inner loop's tilt setpoint, is what produces the actual translational and rotational behavior of the chassis under operator command. The cost of this addition is the extra tuning surface that comes with a second control loop,

including the loop-rate divisor (`VEL_LOOP_DIV`) and a small set of velocity-loop gains that interact with the inner-loop gains during transients. The benefit is qualitatively different behavior: a robot that holds itself up versus a robot that holds itself up *and* can be driven and turned. For a follow-on build, or for any future team starting from a balance-only Instructables-class reference, the encoder-based velocity loop is recommended as essential infrastructure and not as an optional refinement, even though it roughly doubles the controller's tuning surface.

V. Project Outcomes and Impact

The CAMS-Bot project delivered a working self-balancing two-wheeled cargo platform that satisfies the structural and balance components of the proposal's headline 5 kg payload specification, accepts manual operator control through a ROS2 teleoperation stack and an Xbox controller, and demonstrates the cascaded PID control architecture appropriate to the inverted-pendulum form factor. Within the scope it occupies, the delivered system is functional: it balances reliably on residential carpet, responds smoothly to drive and turn commands, holds 5 kg in static balance, and provides operator-initiated motor cutoff through both a hardware kill switch and a serial stop command. The dynamic 5 kg locomotion case is the primary unmet specification, for the geometric reasons analyzed in Section 4.5, and the autonomous-following functional set was descoped with the withdrawal of the project partner responsible for the vision pipeline.

Impact on the target user population. The delivered system is appropriately characterized as an early proof of concept rather than a deployable assistive device. In its current form, the platform demonstrates that a residentially-scaled self-balancing cargo robot can be designed, fabricated, and made to work on an all-maker-space build path, and that a cascaded PID controller running on a single low-cost microcontroller can produce stable balance behavior sufficient to support the assistive task. These are genuine prerequisites for a residential cargo-following device of the kind motivated in Section 1, and demonstrating them on a working build is a non-trivial contribution. However, several specific architectural changes (larger and pneumatic wheels, a metal chassis with a solid drive axle, a cargo platform mounted at or below the axle, and the descoped vision pipeline) are required before the platform could plausibly serve a user in a residential environment. Those changes are developed as concrete future-work items in Section 7. The short-term impact of the project is therefore confined to the proof-of-concept role: it shows that the problem is tractable on a student budget and identifies the specific architectural gaps a follow-on build would need to close. The long-term impact is contingent on those gaps being closed; if they were, the platform class CAMS-Bot demonstrates would address a real and underserved residential need (Section 2).

Differences from the original proposal. The delivered system differs from the proposal in three categories of change. First, several technical refinements were made during implementation that produced a more reliable build than the proposal specified: the L298N motor driver replaced the TB6612FNG, the balance controller was implemented in MicroPython rather than C++, and the Pi 4-to-Pico link was moved from USB CDC serial to hardware UART. These changes are detailed in Section 3.4 and were each driven by an integration problem that surfaced during development. Second, the safety mechanism delivered (an operator-initiated kill switch) is a partial substitute for the proposal's automatic in-loop tilt cutoff. Third, and most significantly, the autonomous-following capability based on the OAK-D Pro and DepthAI was descoped with the partner withdrawal one week before the project deadline. The hardware to support that subsystem (the camera, the camera mount, and the command interface from the high-level node graph to the balance controller) is in place; the vision software that would have driven it is not. This reduces the delivered system from the proposal's autonomous follower to a manually-driven balance platform with the assistive payload-carrying behavior preserved.

VI. Lessons Learned

Unexpected technical challenges. The dominant technical surprise of the project was how much of a self-balancing controller's behavior depends on a small number of non-obvious low-level details. The motor's electrical start threshold was not visible in any motor datasheet specification, and discovering that PWM commands below the start threshold produce no motion at all (silently corrupting closed-loop behavior, since the controller believes it has applied a correction the plant ignored) cost more debugging time than any single classical-controls insight. Similar non-obvious lessons surfaced for the IMU boot sequence (a 2 s startup delay on the MPU6050 is the difference between reliable initialization and intermittent boot failures), the deadband state preservation (zeroing the previous-error and filtered-derivative state at deadband entry causes derivative spikes on re-entry), and the choice of serial transport (USB CDC blocking-read behavior is incompatible with a tight control-loop budget; hardware UART with non-blocking reads solved it cleanly). None of these appears in introductory PID literature, and each had to be discovered the hard way. The geometric tipping behavior of the loaded chassis (Section 4.5) was also unexpected: classical balance-bot tutorials demonstrate balance on lightly-loaded chassis where the cargo geometry never becomes limiting, so the lesson that high-mounted cargo on small wheels makes locomotion under load impossible regardless of gain tuning is not in the standard learning path.

What went well, and why. Three things went well. First, the cascaded PID architecture adapted from the Instructables reference build [16] worked as advertised once the low-level details above were resolved, and the encoder-driven outer velocity loop in particular was instrumental in making the platform actually controllable rather than merely upright (Section 4.5). Second, the split power and compute architectures (12 V motor bus separate from the USB battery logic

supply; the Pico handling deterministic balance separately from the Pi 4's ROS2 stack) avoided entire classes of bug that would have cost real time to diagnose, and the foresight to build this split early in the project paid for itself many times over. Third, the all-in-house CAD-to-fabrication path through the Northeastern FlexLab maker space, with laser-cut acrylic decks, 3D-printed PLA brackets, and PETG only at the highest-load wheel bushing, kept iteration time to hours rather than days; this mattered most during the wheel-bushing convergence cycle.

Decisions that would change in a follow-on build. Three architectural decisions would change. The wheel selection (8-inch polyurethane intended for warehouse flooring) is the single most consequential limiter of the delivered system's behavior, both for the geometric tipping failure under load (Section 4.5) and for the carpet-versus-smooth-floor stability asymmetry. Larger pneumatic wheels would address both. The chassis material (laser-cut acrylic and 3D-printed plastic) is appropriate for the scale of the current build but does not extend to the larger-wheeled platform a follow-on build would need; a metal chassis with a solid drive axle through the chassis admits both the larger scale and the option of mounting the cargo platform at or below the axle, which is the second of the two geometric fixes for the loaded-locomotion failure. Finally, the evaluation infrastructure should have included automated logging of tilt-angle and encoder-velocity traces from the start of tuning, rather than relying on visible chassis behavior; the gain set the project converged to is acceptable but is not maximally tight, and a logged-trace retune would have produced a more quantitatively defensible controller in the same wall-clock time.

Recommendations for future teams. Future student teams attempting a similar build should know that the project is feasible on a near-zero parts budget through maker-space fabrication, but several specific items will save substantial debugging time. Set the minimum effective controller output above the motor's electrical start threshold, not a nominal small value. Delay the first IMU read by a generous margin (2 s on the MPU6050) before starting the control loop. Filter the derivative term and clamp the integral term as a matter of course, not as an optimization. Preserve controller state across deadband transitions; do not zero it. Tune the balance setpoint before introducing K_I, and use the encoder-driven outer loop from the start rather than treating it as an optional refinement. Use hardware UART rather than USB CDC for the link between high-level and low-level boards. Plan the chassis geometry so that the cargo platform is at or below the wheel axle, and select wheels large enough that the geometric tipping envelope under expected loads is comfortable. Finally, log control-loop signals to file from day one of tuning; visible behavior is sufficient to converge a controller, but logged traces are sufficient to converge it well.

Personal learning. The project required learning, from a base of no prior exposure, both classical control theory at the level needed to implement and tune a cascaded PID balance controller, and parametric mechanical CAD at the level needed to design every custom part on the robot. Both domains became genuinely usable through the iterative hardware-in-the-loop work the project required, in a way that classroom-only exposure does not produce. The delivered system is a proof of concept with known and specifically-articulated limitations rather than a deployable device, but it works, and it works well enough on the static portion of its headline specification to demonstrate that the class of platform CAMS-Bot represents is achievable on a student budget by a single author with maker-space access.

VII. Limitations and Future Work

The delivered CAMS-Bot system has well-characterized limitations, identified through the evaluation in Section 4 and the analysis in Section 4.5. This section enumerates them and pairs each with concrete future work that would address it. The items are ordered roughly by impact on the assistive task: hardware geometry first, then control and operator interface, then software completeness.

Limitation: loaded locomotion fails (Task 3b). The current 8-inch wheels and the cargo platform mounted above the wheel axle produce a geometric tipping envelope too small to admit the lean required for loaded driving (Section 4.5). *Future work:* a chassis redesign using larger pneumatic wheels in the 24–29 inch range, a metal frame stiff enough at that scale, and a solid drive axle through the chassis. The solid axle admits two changes that the current motor-mounted-to-deck arrangement does not: a structurally robust drive coupling between motor and wheel that does not depend on a 3D-printed bushing as the load path (see weak-point limitation below), and a cargo platform mounted at or below the axle plane to reduce the moment arm of the loaded center of mass.

Limitation: the wheel-to-motor coupling is the mechanical weak point. The PETG-printed wheel bushings (`Bush_v2.stl`) sit between the FIT0186 motor's flatted D-shaft and the polyurethane wheel bore, and every newton-meter of motor torque passes through this single 3D-printed part. The bushings hold under the loads the current platform applies, but they are the part most likely to fail under the higher torques a heavier loaded build would impose, and a slipping or fatigued bushing produces the worst class of failure (silent loss of drive on one wheel, with the controller continuing to issue commands the chassis can no longer execute). *Future work:* a solid metal axle running through the chassis, with the wheels keyed to the axle and the motors driving the axle through a belt or chain reduction. This replaces the printed bushing as the load-path element and decouples wheel mounting from motor mounting, allowing each to be optimized independently.

Limitation: stability is surface-dependent. The same gain set produces reliable balance on residential carpet but degraded behavior on hardwood and tile (Section 4.4). Two contributing causes were identified: the controller is non-adaptive across surfaces, and the polyurethane wheels were specified for warehouse flooring rather than residential surfaces. *Future work:* pneumatic wheels would provide passive compliance on smooth flooring that the current rigid polyurethane does

not, addressing the wheel-side cause. A gain-scheduled or autotuning controller that adjusts gains in response to measured plant response would address the controller-side cause; a simple version schedules gains based on a surface-detection signal (for example, the variance of the tilt rate during quiet balance), and a more sophisticated version uses the encoder-velocity loop's tracking error to identify plant conditions online.

Limitation: flat ground only. The platform was tested on level surfaces. Inclines and thresholds were not evaluated. Although no specific failure mode on inclines has been observed, the controller is non-adaptive (above), the velocity loop's setpoint biasing assumes a level surface as the natural balance point, and the small wheel diameter does not roll over thresholds well. *Future work:* the larger pneumatic wheels and gain-scheduled controller above are prerequisites for inclined or threshold operation; explicit testing across inclines, hardwood-to-carpet transitions, and small thresholds would then validate the upgraded platform on the surface distribution of an actual residence.

Limitation: throttle control through the Xbox controller is coarse. The current drive-input shaping (Section 3.4 and the `MAX_DRIVE_ANGLE`, `VEL_DRIVE_SCALE`, `VEL_RAMP_RATE`, and `TRIGGER_EDGE` parameters) produces controllable but coarse throttle response: the analog stick maps to lean angle and velocity target without a tunable response curve, and the operator does not have fine control at low speeds. *Future work:* introduce a configurable response curve on the analog input (an exponent or a piecewise-linear curve) so that small stick deflections produce small velocity targets with finer resolution, while full-stick deflection still produces the maximum lean. A second improvement is operator-tunable maximum lean and maximum velocity, exposed as ROS2 parameters on the teleoperation node rather than as firmware constants on the Pico, so that the same firmware can be used across operator preferences without re-flashing.

Limitation: safety relies on operator-initiated cutoff. The delivered safety mechanism is a hardware kill switch (Section 3.1, Section 3.4), which satisfies the operator-in-the-loop case but does not satisfy the proposal's automatic in-loop tilt cutoff for fall-class events that complete faster than an operator can react. *Future work:* an automatic tilt-threshold cutoff in the Pico balance loop. The implementation is small: when the absolute tilt exceeds a threshold (for example, 30°), drive PWM to zero unconditionally and require an explicit operator reset before resuming. This is straightforward to add and is a high-priority safety improvement.

Limitation: the autonomous-following capability was descoped. The OAK-D Pro and DepthAI person-tracking pipeline that the proposal specified was not delivered (Section 3.4). The hardware to support it (camera, mount, USB connection to the Pi 4, and the same drive/turn command interface used by the manual teleoperation node) is in place. *Future work:* a vision and tracking node that runs on the Pi 4 in ROS2, performs YOLO-based person detection on the OAK-D Pro stream, estimates the user's bearing and range relative to the chassis, and publishes drive and turn commands to the Pico through the same UART protocol the manual teleoperation node currently uses. The descoped safety requirements (stop on loss of detection, cap on autonomous-mode speed) would be implemented alongside this node.

Limitation: evaluation was qualitative rather than logged. The metrics in Section 4.4 are largely binary or qualitative, because the project's instrumentation did not include logged tilt-angle, encoder-velocity, or controller-output traces. *Future work:* log these signals to file (or stream them to the Pi 4 over UART for recording) during all subsequent tuning and evaluation. Quantitative metrics such as RMS tilt error, velocity-loop tracking error, and disturbance-rejection time become available once the signals are logged, and the gains in Table III can be tightened against measured performance rather than visual inspection.

VIII. Conclusion

This report described CAMS-Bot, a self-balancing two-wheeled cargo platform developed for the EECE 5552 Assistive Robotics course as a step toward a residentially-scaled cargo-following device for users with mobility limitations. The motivating assistive task is the routine but high-leverage problem of transporting everyday items between rooms, a task that compounds the existing physical burden of users who rely on stability aids. Existing solutions either require the user to push the device (rollators and walkers) or are sized and priced for industrial environments rather than residential use, leaving a gap that CAMS-Bot was designed to address.

The delivered system is built around a Raspberry Pi 4 running ROS2 Jazzy in Docker for high-level control, a Raspberry Pi Pico running a cascaded PID balance controller in MicroPython at 100 Hz, an MPU6050 IMU for tilt sensing, two 12 V DC gearmotors with quadrature encoders driving 8-inch polyurethane wheels through an L298N H-bridge, and a hybrid chassis built from laser-cut acrylic decks, 3D-printed PLA brackets, and PETG bushings at the wheel-to-motor coupling. An Xbox controller provides operator input through a ROS2 teleoperation node that forwards drive and turn commands to the Pico over a hardware UART link.

The platform balances reliably on residential carpet, accepts smooth teleoperated drive and turn commands, and successfully holds the project's headline 5 kg payload in static balance. The dynamic 5 kg locomotion case fails for analyzed geometric reasons, and the autonomous person-following capability was descoped with the withdrawal of the project partner responsible for the vision pipeline. The principal contributions of the project are a working residentially-scaled self-balancing cargo platform built on a near-zero-cost makerspace fabrication path, a fully documented cascaded PID controller including several non-obvious low-level practices (smooth deadband feedforward, state-preserving deadband, stiction-overcome kick, complementary IMU fusion) that are essential to the platform's behavior, and a clear architectural roadmap (Section 7) for the follow-on build that would extend

the platform from proof of concept to deployable assistive device.

REFERENCES

[1] Centers for Disease Control and Prevention, "Disability impacts all of us infographic," https://www.cdc.gov/disability-and-health/articles-documents/disability-impacts-all-of-us-infographic.html, July 2024, national Center on Birth Defects and Developmental Disabilities. Accessed: 2026-03-09.

[2] Disabled World, "Mobility device statistics: United States," https://www.disabled-world.com/disability/statistics/mobility-stats.php, October 2022, accessed: 2026-03-09.

[3] T. L. Chen, M. Ciocarlie, S. Cousins, P. M. Grice, K. Hawkins, K. Hsiao, C. C. Kemp, C.-H. King, D. A. Lazewatsky, A. E. Leeper *et al.*, "Robots for humanity: using assistive robotics to empower people with disabilities," *IEEE Robotics & Automation Magazine*, vol. 20, no. 1, pp. 30–39, 2013.

[4] P. Balatti, I. Ozdamar, D. Sirintuna, L. Fortini, M. Leonori, J. M. Gandarias, and A. Ajoudani, "Robot-assisted navigation for visually impaired through adaptive impedance and path planning," 2023. [Online]. Available: https://arxiv.org/abs/2310.14705

[5] A. Deb, Z. Wypych, J. Lonner, and H. Ashrafiuon, "Design and control of an autonomous robot for mobility-impaired patients," *IFAC-PapersOnLine*, 2021. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2405896321022254

[6] S. A. Olatunji, J. S. Shim, A. Syed, Y.-L. Tsai, A. E. Pereira, H. P. Mahajan, R. A. Mudar, and W. A. Rogers, "Robotic support for older adults with cognitive and mobility impairments," *Frontiers in Robotics and AI*, 2025. [Online]. Available: https://www.frontiersin.org/journals/robotics-and-ai/articles/10.3389/frobt.2025.1545733/full

[7] A. Steinfeld, T. Fong, D. Kaber, M. Lewis, J. Scholtz, A. Schultz, and M. Goodrich, "Common metrics for human-robot interaction," in *Proceedings of the 1st ACM SIGCHI/SIGART conference on Human-robot interaction*, 2006, pp. 33–40.

[8] S. G. Hart, "Nasa-task load index (nasa-tlx); 20 years later," in *Proceedings of the human factors and ergonomics society annual meeting*, vol. 50, no. 9. Sage publications Sage CA: Los Angeles, CA, 2006, pp. 904–908.

[9] J. R. Lewis, "Measuring perceived usability: The csuq, sus, and umux," *International Journal of Human–Computer Interaction*, vol. 34, no. 12, pp. 1148–1156, 2018.

[10] Foxtech, "FOLO-200 heavy duty 100kg four-wheel cargo trolley automatic human-following cart robot," https://store.foxtech.com/folo-200-heavy-duty-100kg-four-wheel-cargo-trolley-automatic-human-following-cart-robot/, 2024, foxtech Store. Accessed: 2026-03-09.

[11] KTLA, "Food delivery robot goes rogue, causes property damage at East Hollywood home," https://ktla.com/news/local-news/food-delivery-robot-goes-rogue-causes-property-damage-at-east-hollywood-home/, 2023, kTLA 5 News. Accessed: 2026-03-09.

[12] ABC7, "Man with cerebral palsy repeatedly hit by delivery robot in West Hollywood," https://abc7.com/post/delivery-robot-hits-man-cerebral-palsy-mobility-scooter-west-hollywood/17866465/, September 2023, accessed: 2026-03-09.

[13] FOX 11 Digital Team, "Food delivery robot, ambulance collide in Hollywood," https://www.foxla.com/news/food-delivery-robot-ambulance-collide-in-hollywood, January 2026, fOX 11 Los Angeles. Accessed: 2026-03-09.

[14] R. Petcu and G.-D. Andreescu, "Two-wheeled inverted-pendulum self-balancing robot with LQR or PID control," in *2022 IEEE Conference*, 2022. [Online]. Available: https://ieeexplore.ieee.org/document/9922862/

[15] S. Saha *et al.*, "Epersist: A two-wheeled self balancing robot using PID controller and deep reinforcement learning," in *2022 IEEE 19th India Council International Conference (INDICON)*, 2022. [Online]. Available: https://ieeexplore.ieee.org/document/10003940/

[16] Instructables, "Arduino based self-balancing bot," https://www.instructables.com/Arduino-Based-Self-Balancing-Bot/, 2024, accessed: 2026-03-09.